

PRD — Smart Fuel Station Monitoring & Management System

Product name: SmartFuel / FuelWatch

Product type: Real-time Fuel Tank Monitoring & Station Management Platform

Primary users: Petrol station owners, managers, supervisors, finance teams, operations teams

Core integrations: Fuel probes + GPS/telematics devices + backend API + email notifications

1. Product Vision

Build a **modern, real-time fuel monitoring platform** that allows petrol station owners and operators to monitor every station, tank, fuel level, refill, consumption event, anomaly, and alert from one centralized system.

The system should turn raw fuel-probe data into **simple, actionable information**.

The experience should feel like a premium SaaS product:

- Awwwards-level visual design
- Clean and minimal
- Extremely easy to navigate
- Data-rich without feeling complicated
- Real-time
- Responsive
- Professional enough for enterprise customers
- Optimized for desktop but fully usable on tablets/mobile

The system should make it possible for a manager to answer within seconds:

How much fuel do I have right now?

Which station is running low?

Which tank received fuel today?

How much fuel was consumed?

When did the fuel movement happen?

Is there anything abnormal happening?

2. Core System Concept

The platform connects:

FUEL PROBE



DEVICE / TELEMETRY API



BACKEND



FUEL MONITORING ENGINE



DATABASE



WEB APPLICATION



DASHBOARD / REPORTS / ALERTS

For each station:

Station

|

|— Address / GPS Location

|

|— Operating Information

|

|— Tanks

|

|— Petrol Tank A

|

|— Petrol Tank B

|

|— Diesel Tank A

|

|— Diesel Tank B

|

|— Fuel Probes

|

|— Fuel Transactions

|

|— Alerts

|

|— Reports

3. Core Objectives

The system must:

1. Monitor fuel levels in real time.
2. Support multiple petrol stations.
3. Support multiple tanks per station.
4. Support multiple fuel types.
5. Receive data from fuel probes.
6. Display current tank levels.
7. Detect refilling.
8. Detect fuel consumption.
9. Calculate fuel movement.
10. Display historical fuel data.
11. Provide fuel usage replay.
12. Generate alerts.
13. Generate automated reports.
14. Allow PDF and Excel exports.
15. Support multiple users.
16. Support role-based permissions.
17. Maintain an audit trail.
18. Provide station-level and organization-level dashboards.
19. Work even when individual devices temporarily lose connectivity.

20. Clearly distinguish real probe readings from calculated/estimated values.

4. UX DESIGN PRINCIPLES

The UI should **not look like a traditional fuel-management system**.

Avoid:

- Crowded tables everywhere
- Excessive borders
- Too many colors
- Complicated menus
- Tiny text
- Technical terminology exposed unnecessarily
- Graphs for information that could be understood more easily visually

Instead use:

- Large information hierarchy
- Cards
- Clear status indicators
- Beautiful typography
- Subtle animations
- Spacious layouts
- Interactive charts
- Underground tank visualizations
- Real-time indicators
- Progressive disclosure
- Contextual actions

Design inspiration

The visual quality should feel comparable to modern products such as:

- Linear
- Stripe

- Vercel
- Apple
- Notion
- Tesla dashboards
- modern fintech dashboards

But the system must retain its own identity.

5. APPLICATION STRUCTURE

Main navigation:

Dashboard

Stations

└─ All Stations

└─ Add Station

Tanks

└─ All Tanks

└─ Tank Configuration

Fuel Movement

└─ Refueling

└─ Consumption

└─ Transactions

Alerts

Reports

└─ Daily

- └— Weekly
- └— Monthly
- └— Custom
- └— Scheduled Reports

Devices

- └— Fuel Probes
- └— GPS Devices
- └— Device Health

Users & Roles

Settings

- └— Organization
- └— Fuel Types
- └— Alert Rules
- └— Notifications
- └— System Settings

Audit Logs

6. GLOBAL DASHBOARD

The main dashboard should immediately communicate the overall state of the business.

Header

Display:

Good morning, George

Organization selector:

PUMA Tanzania ▼

Date/time:

25 September 2026 · 15:42

Real-time indicator:

 **Live**

7. TOP-LEVEL KPI CARDS

Display:

Total Stations

12

11 Online · 1 Offline

Total Fuel Available

486,240 L

Across all monitored tanks.

Today's Consumption

42,840 L

Today's Refills

67,500 L

Active Alerts

4

Offline Devices

1

8. STATION OVERVIEW

Display all stations in an elegant grid/list.

Example:

PUMA Arusha — Main Branch

Online

Fuel:

82,450 L

Capacity:

120,000 L

Utilization:

68.7%

Today's consumption:

8,420 L

Today's refills:

15,000 L

Alerts:

1

Location:

Arusha

Button:

View Station →

Station status

Use:

-  Online
-  Warning
-  Critical
-  Offline

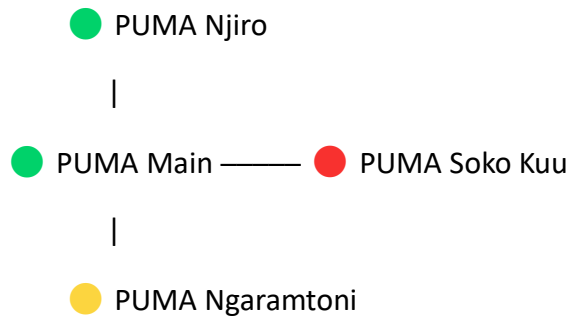
Do not rely solely on colors; include icons/text for accessibility.

9. MAP VIEW

Provide an optional map view showing all stations.

Each station appears as a marker.

Example:



Clicking a station marker opens:

- Station name
- Current fuel
- Tank count
- Current alerts
- Device status
- View Station

10. STATION DETAILS PAGE

When the user selects:

PUMA Arusha — Main Branch

Show:

PUMA Arusha — Main Branch

 Arusha, Tanzania

- All systems operational

Last updated:

15:42:21

Actions:

- Add Tank
- Edit Station
- Export Report
- Configure Alerts

11. STATION SUMMARY

Cards:

Total Fuel

82,450 L

Capacity

120,000 L

Available

68.7%

Today's Consumption

8,420 L

Today's Refills

15,000 L

Active Alerts

1

12. MULTIPLE TANK SUPPORT

A station may have unlimited tanks.

Example:

PUMA Arusha

PETROL

Tank A

Tank B

DIESEL

Tank A

Tank B

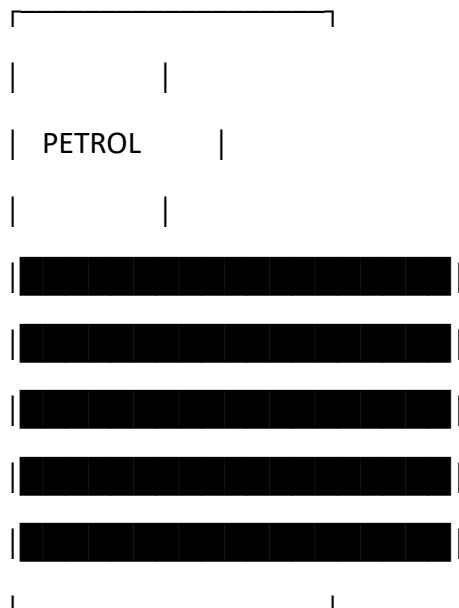
Tank C

Each tank gets its own monitoring profile.

13. TANK CARD

Each tank should be visually represented as an **underground fuel tank**, not simply a progress bar.

Example:



Tank A

38,420 L / 50,000 L

76.8%

The tank should have a subtle liquid animation.

Tank states

Full:

85–100%

Normal:

30–84%

Low:

15–29%

Critical:

<15%

These thresholds should be configurable.

14. TANK DETAILS PAGE

Click:

Petrol Tank A

The page becomes the main monitoring interface for that tank.

Header:

Petrol Tank A

PUMA Arusha — Main Branch

- LIVE

Last reading:

15:42:21

15. TANK LIVE INFORMATION

Display:

Current Level

38,420 L

Capacity

50,000 L

Remaining Capacity

11,580 L

Fill Percentage

76.8%

Today's Consumption

4,820 L

Today's Refilling

10,000 L

Estimated Available

38,420 L

16. PROBE DATA

The frontend should expose all meaningful data received from the probe.

Depending on what the hardware API provides, support:

- Fuel volume
- Fuel level
- Fuel height
- Temperature
- Water level
- Product level

- Tank capacity
- Product density
- Device signal
- Device battery
- Device status
- Timestamp
- Communication status
- Probe ID
- Sensor status

Important:

The system must never invent probe values.

If the probe does not provide a particular measurement, the UI should show:

Not available

rather than a fake value.

17. REAL-TIME UPDATE ENGINE

The tank page should automatically update.

Example:

15:42:21

38,420 L

15:42:31

38,415 L

15:42:41

38,407 L

The user should not need to refresh the page.

Show:

 Live data

If data becomes stale:

 Last updated 2 minutes ago

If connection is lost:

 Device offline

18. REFILL DETECTION

The system should automatically detect refilling.

Example:

14:03

24,200 L

14:04

24,850 L

14:05

25,700 L

14:06

27,100 L

System identifies:

Fuel Refill Detected

Amount:

2,900 L

Start:

14:03

End:

14:06

Duration:

3 minutes

19. REFILL EVENT

Create a dedicated event screen.

 REFILL DETECTED

Petrol Tank A

+2,900 L

14:03 — 14:06

PUMA Arusha

Provide:

- Starting level
 - Ending level
 - Added volume
 - Duration
 - Date
 - Station
 - Tank
 - Probe
 - Event confidence/status
-

20. CONSUMPTION DETECTION

When the fuel level decreases, the system should identify consumption.

Example:

10:20

32,000 L

10:21

31,980 L

10:22

31,950 L

Display:

Fuel Consumption

-50 L

Timestamp:

10:22

Tank:

Petrol Tank A

21. FUEL MOVEMENT TIMELINE

Every significant movement should be recorded.

Example:

TODAY

15:42

-20 L

Fuel consumption

15:40

-35 L

Fuel consumption

14:03

+2,900 L

Refill

13:20

-120 L

Fuel consumption

Clicking an event opens details.

22. FUEL USAGE REPLAY

This is one of the system's signature features.

User selects:

Replay period

Today

Last 24 hours

Last 7 days

Last 30 days

Custom

The tank visualization then animates historically.

Example:

DAY 1

42,000 L

↓ consumption

DAY 2

38,500 L

↓ consumption

DAY 3

35,200 L

↑ refill

DAY 4

47,000 L

The animation should have:

- Play
- Pause
- Speed
- Timeline
- Timestamp
- Current fuel level
- Movement amount

Example:

▶ 1× 2× 4×

This should feel similar to a playback interface.

23. HISTORICAL CHART

Provide a detailed chart beneath the tank.

Selectable:

- Fuel Level
- Consumption
- Refills

- Temperature
- Water Level

Time:

- 24 hours
- 7 days
- 30 days
- 90 days
- Custom

Hovering over the chart:

15:32

Fuel Level

38,420 L

Movement

-35 L

24. FUEL TYPES

Default fuel types:

- Petrol
- Diesel

Allow administrators to add custom types.

Each fuel type should have:

System name

Petrol

Display name

Unleaded Petrol

Display name is optional.

Example:

Fuel Type

Petrol

Display name

Unleaded 95

The system should support:

- Petrol
 - Diesel
 - Premium Petrol
 - Kerosene
 - Custom fuels
-

25. STATION CREATION

Create Station form:

Basic information

- Station name
- Station code
- Address
- City
- Region
- Country
- Phone
- Email

Location

- Latitude
- Longitude
- Map selector

- GPS coordinates

Operating information

- Opening time
- Closing time
- Time zone

Configuration

- Currency
 - Default measurement unit
 - Alert thresholds
-

26. TANK CREATION

Create Tank:

Tank information

- Tank name
- Tank code
- Fuel type
- Optional fuel display name
- Tank capacity
- Minimum level
- Low level threshold
- Critical level threshold

Physical information

- Tank type
- Underground / above-ground
- Manufacturer
- Installation date

Device

- Probe

- Probe ID
 - Device serial number
-

27. DEVICE MANAGEMENT

Dedicated device management section.

Fuel Probe

Display:

Probe ID

PROBE-001829

Station

PUMA Arusha

Tank

Petrol Tank A

Status

- Online

Last communication

15:42:21

Signal

Excellent

Device status:

- Online
- Delayed
- Offline

- Fault
 - Never connected
-

28. GPS DEVICE INTEGRATION

Support GPS/telematics devices separately.

A GPS device can be associated with:

- Station
- Vehicle
- Organization

Potential future use:

- Fuel delivery vehicle tracking
 - Delivery verification
 - Fleet monitoring
 - Driver activity
-

29. ALERT SYSTEM

The alert system is a major feature.

Alert categories:

Low fuel

⚠ Petrol Tank A

Fuel level has dropped below 20%.

Current:

9,800 L

Critical fuel

● CRITICAL

Petrol Tank A

Fuel level below 10%.

Current:

4,700 L

Refill detected

 REFILL DETECTED

+12,500 L

Unexpected fuel loss

 UNUSUAL FUEL LOSS

Petrol Tank A

Estimated loss:

450 L

Probe offline

 PROBE OFFLINE

No data received for 10 minutes.

Communication failure

Water detected

If supported by the probe.

Rapid level change

Overfill

Temperature abnormality

If temperature monitoring is available.

30. ALERT CENTER

Dedicated page:

Alerts

All | Critical | Warning | Resolved

 Critical

Petrol Tank A below critical level

 Warning

Diesel Tank B probe communication delayed

 Refill

Petrol Tank B +12,000 L

✓ Resolved

Probe connection restored

31. ALERT ACTIONS

Users should be able to:

- Acknowledge
- Resolve
- Assign
- Add note
- View tank
- View event
- Export

Maintain alert history.

32. NOTIFICATIONS

Support:

In-app

Real-time notification center.

Email

Send:

- Low fuel
- Critical fuel
- Refill
- Unexpected loss
- Device offline
- Daily summary

Future:

- SMS
 - WhatsApp
 - Push notifications
-

33. REPORTING ENGINE

Reports should be extremely powerful but simple.

Report categories:

Daily Fuel Report

Include:

- Opening level
- Refills
- Consumption
- Closing level

- Variance
- Alerts

Weekly Report

Monthly Report

Custom Date Report

User chooses:

From:

01 Sept 2026

To:

25 Sept 2026

34. REPORT FILTERS

Filter by:

- Organization
 - Station
 - Tank
 - Fuel type
 - Date
 - Event type
 - User
 - Device
-

35. REPORT EXPORT

Export:

PDF

Excel

Potential future:

CSV

Report should contain:

- Company logo
 - Report title
 - Date range
 - Station
 - Tank
 - Generated date
 - Generated by
 - Summary
 - Detailed events
 - Charts
 - Totals
-

36. AUTOMATED REPORTS

Users can schedule reports.

Example:

Daily Fuel Report

Recipients:

manager@example.com

finance@example.com

Schedule:

Every day

Time:

18:00

Format:

PDF

Supported:

- Daily
- Weekly
- Monthly
- Custom

37. CONSUMPTION ANALYTICS

Provide analytics such as:

Average daily consumption

4,820 L/day

Highest consumption day

18 Sept

Average refill

8,500 L

Current stock coverage

6.4 days

Where appropriate, calculated metrics should clearly be labeled as **estimates/calculations**, not probe measurements.

38. STOCK COVERAGE

Calculate:

Current fuel

÷

Average daily consumption

=

Estimated days remaining

Example:

38,420 L

Average consumption:

5,200 L/day

Estimated:

7.4 days

Display:

Estimated stock coverage: 7.4 days

39. INVENTORY RECONCILIATION

Add an important operational feature:

Compare:

Opening Stock

+

Refills

-

Consumption

=

Expected Closing Stock

Compare expected vs measured probe level.

Example:

Expected:

38,600 L

Probe:

38,420 L

Variance:

-180 L

This can generate an alert if variance exceeds a configurable threshold.

40. ANOMALY DETECTION

The system should identify unusual patterns.

Examples:

- Sudden large fuel decrease
- Fuel decrease outside operating hours
- Refill followed immediately by abnormal loss
- Probe readings jumping unrealistically
- Repeated communication interruptions
- Tank level exceeding capacity
- Tank level dropping unusually fast

Display:

Possible anomaly detected

Rather than automatically claiming theft or fraud.

41. OPERATING HOURS

Each station can configure operating hours.

Example:

Monday

06:00 – 23:00

Tuesday

06:00 – 23:00

This helps identify unusual fuel movement outside normal operating periods.

42. USER MANAGEMENT

Support multiple users.

Example:

Owner

Manager

Station Manager

Supervisor

Finance

Viewer

43. ROLE PERMISSIONS

Permissions should be granular.

Example:

Permission	Owner	Manager	Viewer
View Dashboard	✓	✓	✓
View Tanks	✓	✓	✓
Add Station	✓	✓	—
Add Tank	✓	✓	—
Edit Station	✓	✓	—
Reports	✓	✓	✓
Export	✓	✓	Optional

Permission	Owner	Manager	Viewer
------------	-------	---------	--------

Manage Users	✓	✓	—
--------------	---	---	---

System Settings	✓	—	—
-----------------	---	---	---

Delete Station	✓	—	—
----------------	---	---	---

Allow custom roles later.

44. MULTI-TENANT ARCHITECTURE

The system should support multiple organizations.

Example:

Organization A

|

├— PUMA Arusha

├— PUMA Ngaramtoni

├— PUMA Njiro

└— PUMA Soko Kuu

Organization B

|

├— Station 1

├— Station 2

└— Station 3

Users should only see resources they have permission to access.

45. AUDIT LOG

Record important actions.

Example:

George added Petrol Tank B

25 Sept 2026

14:42

Sarah changed low-fuel threshold

25 Sept 2026

13:20

Track:

- User
 - Action
 - Resource
 - Previous value
 - New value
 - IP/device where appropriate
 - Timestamp
-

46. SEARCH

Global search:

⌘ K

Search:

- Stations
- Tanks
- Probes
- Alerts
- Users
- Reports

Example:

User types:

Petrol Tank A

System immediately shows:

Tank

Petrol Tank A

PUMA Arusha

38,420 L

47. COMMAND PALETTE

A premium UX feature.

Press:

Ctrl + K

Actions:

Search station

Add station

Add tank

View alerts

Generate report

Find tank

Manage users

48. RESPONSIVE DESIGN

Desktop:

Primary experience.

Tablet:

Fully functional.

Mobile:

Optimized for monitoring.

Mobile dashboard should prioritize:

1. Alerts
2. Fuel levels
3. Station status
4. Tank status
5. Recent movement

49. REAL-TIME UX

When new probe data arrives:

Do not abruptly refresh the whole page.

Instead:

- Animate changed number
- Subtly animate tank liquid
- Update timestamp
- Add event to timeline
- Update charts

Example:

38,420 L → 38,405 L

Use subtle number transitions.

50. TANK ANIMATION

The underground tank should be a signature visual component.

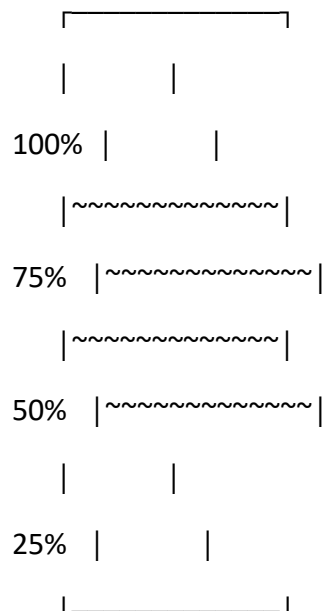
Requirements:

- Liquid level changes smoothly
- Slight liquid movement
- Capacity markings

- Current level
- Percentage
- Fuel type
- Tank name

Example:

PETROL



38,420 L

76.8%

Avoid excessive animation because this is an operational system.

51. DESIGN SYSTEM

Typography

Use a premium modern font such as:

- Inter
- Geist
- Manrope

Layout

Large whitespace.

12-column desktop grid.

Cards with subtle radius.

Avoid excessive shadows.

Colors

Primary:

Neutral dark/white foundation.

Fuel-specific colors can be used sparingly.

Status:

Green = healthy

Amber = warning

Red = critical

Blue = information

Gray = inactive

52. DARK MODE

Support:

Light

Dark

System

Dark mode should not simply invert colors.

Charts, tank visualizations and alerts need dedicated dark-mode styling.

53. EMPTY STATES

Every empty screen needs a useful state.

Example:

No tanks yet

Add your first tank to start monitoring
fuel levels in this station.

[+ Add Tank]

54. ERROR STATES

Example:

Unable to load tank data

The monitoring service is temporarily unavailable.

[Try Again]

Never show raw API errors to users.

55. DEVICE FAILURE UX

If probe stops communicating:

⚠ Data may be outdated

Last reading:

14:42

Current time:

15:02

Probe:

Offline

Do not display the old value as if it were current.

56. BACKEND ARCHITECTURE

Recommended architecture:

Frontend

Next.js / React



API Layer



Backend

Node.js / NestJS

or equivalent



Database

PostgreSQL



Real-Time Layer

WebSocket / Server-Sent Events



Device Integration Layer

Tectonic / Probe API

GPS API

57. DEVICE INTEGRATION LAYER

The backend should not tightly couple the application to one hardware vendor.

Create an abstraction:

Device Provider



normalizeReading()



Fuel Monitoring Engine

This allows future support for:

- Tectonic
 - Other fuel probes
 - Different GPS providers
 - Different IoT platforms
-

58. NORMALIZED READING MODEL

Every device reading should eventually become something like:

```
{  
  "stationId": "...",  
  "tankId": "...",  
  "deviceId": "...",  
  "timestamp": "...",  
  "fuelType": "petrol",  
  "volumeLiters": 38420,
```

```
"levelPercent": 76.8,  
"temperature": 28.4,  
"waterLevel": 0,  
"signal": "good"  
}
```

The actual fields should depend on the device API.

59. EVENT ENGINE

The backend should convert readings into meaningful events.

RAW READING



COMPARE PREVIOUS READING



DETERMINE MOVEMENT



CLASSIFY EVENT



STORE EVENT



UPDATE DASHBOARD



TRIGGER ALERT IF NECESSARY

Possible events:

CONSUMPTION

REFILL

LOW_LEVEL

CRITICAL_LEVEL

OVERFILL

WATER_DETECTED

DEVICE_OFFLINE

DEVICE_ONLINE

ANOMALY

60. DATA STORAGE

Store raw readings separately from interpreted events.

Raw readings

Immutable device data.

Derived events

System interpretation.

This is important because calculations/rules may change in the future.

61. DATABASE ENTITIES

At minimum:

Organization

User

Role

Permission

Station

StationAddress

FuelType

Tank

Device

Probe

GPSDevice

Reading

FuelMovement

RefillEvent

ConsumptionEvent

Alert

Notification

Report

ScheduledReport

AuditLog

62. SECURITY

Implement:

- Secure authentication
 - Password hashing
 - Session management
 - MFA-ready architecture
 - Role-based access control
 - Tenant isolation
 - API authentication
 - Device authentication
 - Rate limiting
 - Input validation
 - Audit logging
 - Secure secrets management
-

63. API REQUIREMENTS

Example endpoints:

POST /auth/login

GET /dashboard

GET /stations

POST /stations

GET /stations/:id

POST /stations/:id/tanks

GET /tanks/:id

GET /tanks/:id/readings

GET /tanks/:id/movements

GET /tanks/:id/replay

GET /alerts

POST /alerts/:id/acknowledge

GET /reports

POST /reports

GET /devices

POST /devices

64. DEVICE WEBHOOK

Where supported:

POST /webhooks/device/:provider

The system receives probe readings.

Validate:

- Device identity
- Signature/API key
- Timestamp

- Payload
- Tank mapping

Then store the reading.

65. DATA VALIDATION

Reject or flag impossible readings.

Examples:

Fuel = -500 L

or:

Tank capacity = 50,000 L

Reading = 80,000 L

These should not silently enter normal reporting.

66. OFFLINE HANDLING

If device data stops arriving:

System should:

1. Detect timeout.
 2. Mark device offline.
 3. Notify appropriate users.
 4. Preserve last valid reading.
 5. Clearly display stale timestamp.
 6. Restore automatically when data returns.
 7. Record outage duration.
-

67. PERFORMANCE

Dashboard should load quickly.

Target:

- Initial dashboard render: <2 seconds where infrastructure allows

- Station switching: near instant
- Tank updates: real time
- Report generation: asynchronous for large reports

Use pagination for large datasets.

68. REPORT GENERATION

For large reports:

Generating report...



Your report will be ready shortly.

Do not freeze the interface.

69. NOTIFICATION CENTER

Top-right bell icon.

Example:

Notifications

 Critical fuel

PUMA Njiro — Diesel Tank B

8 minutes ago

 Refill detected

PUMA Arusha — Petrol Tank A

22 minutes ago

 Probe offline

PUMA Soko Kuu

1 hour ago

[View all notifications](#)

70. USER ONBOARDING

First login:

Welcome to SmartFuel

Let's set up your organization.

Step 1

Organization

Step 2

Add your first station

Step 3

Add tanks

Step 4

Connect fuel probes

Step 5

Configure alerts

You're ready.

71. STATION SETUP WIZARD

Use a guided process rather than a huge form.

1. Station



2. Location



3. Tanks



4. Fuel Types



5. Devices



6. Alerts



7. Finish

72. IMPORTANT UX FEATURE: QUICK ACTIONS

Dashboard should include:

+ Add Station

+ Add Tank

Connect Device

Generate Report

View Alerts

73. IMPORTANT UX FEATURE: CONTEXTUAL ACTIONS

When viewing a tank:

Tank A

[View Replay]

[Generate Report]

[Configure Alerts]

[Edit Tank]

Do not make users navigate through Settings for common actions.

74. STATION COMPARISON

Allow managers to compare stations without ranking them.

Example:

Station	Fuel	Consumption	Refills
PUMA Arusha	82,450 L	8,420 L	15,000 L
PUMA Njiro	64,210 L	7,200 L	10,000 L
PUMA Ngaramtoni	42,100 L	4,800 L	8,000 L

75. TANK COMPARISON

Similarly:

Petrol Tank A

76.8%

Petrol Tank B

54.2%

Diesel Tank A

82.1%

76. FUEL MOVEMENT LEDGER

A complete searchable transaction history.

Columns:

Timestamp

Station

Tank

Fuel

Event

Volume

Level Before

Level After

Device

Status

Example:

14:03

PUMA Arusha

Petrol A

REFILL

+2,900 L

24,200 → 27,100

Probe 001

Confirmed

77. DATE RANGE UX

Use a beautiful date picker.

Quick options:

Today

Yesterday

Last 7 days

Last 30 days

This month

Last month

Custom range

All relevant charts and metrics update together.

78. REAL-TIME DATA INDICATOR

Always communicate whether data is actually live.

Examples:

 **LIVE**

Updated 3 seconds ago

 **DELAYED**

Updated 4 minutes ago

 **OFFLINE**

Last reading 28 minutes ago

This is critical for user trust.

79. DATA CONFIDENCE

For calculated fuel movements, optionally display:

Movement detected

Confidence: High

This is useful because real-world probes may have noise.

80. CONFIGURABLE THRESHOLDS

Admins should configure:

- Low fuel %
- Critical fuel %
- Overfill %

- Unexpected movement
 - Offline timeout
 - Anomaly sensitivity
 - Operating hours
-

81. INTERNATIONALIZATION

Prepare the system for:

- English
- Swahili

Potential future:

- French
- Arabic

Do not hard-code UI strings.

82. UNITS

Support:

- Liters
- Gallons

Temperature:

- Celsius
- Fahrenheit

Currency:

- TZS
- configurable currencies

Default should be configurable by organization.

83. ACCESSIBILITY

Follow WCAG principles.

Ensure:

- Keyboard navigation
- Screen-reader labels
- Good contrast
- Visible focus states
- No color-only status communication
- Accessible charts
- Accessible modals/forms

84. MOBILE EXPERIENCE

Mobile bottom navigation:

Home

Stations

Alerts

Reports

More

Tank page:

Petrol Tank A



76.8%

38,420 L

Today's use

4,820 L

Refilled

10,000 L

85. PWA SUPPORT

Consider making the application installable as a Progressive Web App.

Users could install:

SmartFuel

on mobile/tablet.

86. FUTURE FEATURES

Design architecture to eventually support:

Fuel dispenser integration

Connect dispenser data so:

Tank movement

↓

Dispenser

↓

Pump

↓

Nozzle

↓

Liters sold

This would allow actual sales reconciliation.

Vehicle/fleet integration

Connect GPS vehicles with fuel consumption.

Fuel delivery management

Track:

Supplier

Delivery vehicle

Expected liters

Delivered liters

Tank received

Variance

Supplier management

Purchase orders

Invoice management

Predictive stock planning

87. IMPORTANT DISTINCTION: TANK CONSUMPTION VS DISPENSER SALES

The system should clearly distinguish:

Tank movement

from

Actual dispenser sales

If only the probe is available, the system can observe a decrease in tank volume but should **not label it "liters sold" unless dispenser/POS data confirms sales.**

Therefore use:

Fuel Consumption / Tank Outflow

until dispenser integration exists.

When dispenser integration is available:

Fuel Sold

This distinction will make the system much more technically reliable.

88. DASHBOARD INFORMATION HIERARCHY

The dashboard should answer questions in this order:

1. Is everything okay?

Station/device status.

2. How much fuel do I have?

Current inventory.

3. What changed?

Refills and consumption.

4. Is there a problem?

Alerts/anomalies.

5. What happened over time?

Charts/replay.

6. What should I report?

Reports/export.

89. DESIGN THE "WOW" MOMENT

The system should have one distinctive visual interaction:

The Tank View

When opening a tank:

1. Tank smoothly appears.
2. Fuel level animates to current level.
3. Current liters appear.
4. Live indicator pulses subtly.
5. Recent movement appears on the timeline.
6. Chart loads below.
7. User can press **Replay** and watch fuel movement over time.

This becomes the product's signature interaction.

90. DESIGN RULE FOR AI AGENT

Do not build a generic admin dashboard.

Build a **premium fuel intelligence platform**.

The interface should communicate:

Real-time fuel visibility.

Every screen should prioritize clarity over information density.

91. MVP PHASE

The first production version should include:

Authentication

- ✓ Login
- ✓ Logout
- ✓ Password reset

Organization

- ✓ Organization
- ✓ Users
- ✓ Roles

Stations

- ✓ Create
- ✓ Edit
- ✓ View
- ✓ Location

Tanks

- ✓ Create
- ✓ Edit
- ✓ Multiple tanks
- ✓ Fuel types

Devices

- ✓ Probe registration
- ✓ Device mapping
- ✓ Device status

Monitoring

- ✓ Real-time readings
- ✓ Fuel level
- ✓ Tank visualization

✓ Tank capacity

✓ Timestamp

Events

✓ Consumption

✓ Refills

✓ Movement history

Alerts

✓ Low fuel

✓ Critical fuel

✓ Device offline

✓ Refill

Analytics

✓ Current inventory

✓ Consumption

✓ Refills

✓ Historical charts

✓ Date ranges

Reports

✓ Daily

✓ Weekly

✓ Monthly

✓ Custom

✓ PDF

✓ Excel

92. PHASE 2

Add:

- Advanced anomaly detection
- Scheduled reports
- SMS
- WhatsApp

- Advanced analytics
 - Station comparison
 - Tank comparison
 - Inventory reconciliation
 - Advanced audit logs
 - Advanced device health
-

93. PHASE 3

Add:

- Dispenser integration
 - POS integration
 - Vehicle/fleet integration
 - Fuel delivery management
 - Supplier management
 - Predictive stock forecasting
 - Automated replenishment recommendations
-

94. ACCEPTANCE CRITERIA

The system should not be considered complete unless:

Station

A user can create a station with:

- Name
- Address
- GPS location
- Multiple tanks

Tank

A user can create:

Petrol Tank A

50,000 L

and associate a probe.

Device

The system receives probe readings and displays them automatically.

Real-time

When the probe changes from:

30,000 L

to:

29,950 L

the dashboard updates without manual refresh.

Refill

When the system detects a substantial positive level change, it creates a refill event.

Consumption

When fuel decreases, the system records the movement.

Historical

User can select:

Last 7 days

and see historical fuel movement.

Replay

User can play the seven-day tank history.

Alert

If the tank reaches configured low level:

An alert is created.

Reporting

User can generate:

PDF + Excel.

Users

Owner can create a sub-user and restrict access.

Offline

If a probe stops communicating, the system clearly shows:

Offline + last reading timestamp.

95. FINAL AI DESIGN/CODING INSTRUCTION

Give the entire system a **premium, Awwwards-level SaaS experience**.

Do not make it look like an old enterprise fuel-management application.

Visual priorities

1. Clarity

Users should understand the system immediately.

2. Real-time feeling

Data should feel alive.

3. Beautiful tank visualization

The underground tank should be a signature component.

4. Excellent typography

Use a modern professional font.

5. Motion

Use subtle transitions and micro-interactions.

6. Responsive

Desktop-first but excellent on mobile.

7. Consistency

Use a proper design system and reusable components.

8. Accessibility

Never communicate status through color alone.

9. Performance

Do not sacrifice speed for visual effects.

10. Trust

Always show timestamps and distinguish live, delayed, offline, measured, and calculated data.

Recommended primary navigation

LOGO		Search...		User
Dashboard		MAIN CONTENT		
Stations				
Tanks				
Fuel Movement				
Alerts				
Reports				
Devices				
Users				
Settings				

The core user journey

LOGIN



DASHBOARD



SELECT STATION



SEE ALL TANKS



SELECT TANK



LIVE TANK VISUALIZATION



CURRENT FUEL



REFILLS / CONSUMPTION



HISTORICAL CHART



FUEL REPLAY



GENERATE REPORT

The most important design principle is:

A user should be able to go from "How much fuel do we have?" to the exact tank, exact fuel movement, exact timestamp, and historical explanation in no more than a few clicks.

This PRD is also structured so the **frontend, backend, database, device-integration layer, reporting engine, alert engine, and future dispenser/POS integration can be developed as one coherent platform rather than bolted together later.**